

Sistemas Operativos II

Práctica 4 – Productor-Consumidor con paso de mensajes

Grado en Ingeniería Informática – USC – Curso 2025-2026

Yago Regueira y Xian Shanti

1. Introducción

El objetivo de este ejercicio es implementar el problema del productor-consumidor utilizando paso de mensajes POSIX entre dos procesos independientes. El productor (prod.c) lee caracteres de un fichero de texto y los envía al consumidor (cons.c), que los almacena en un fichero de salida. Ambos procesos se comunican exclusivamente a través de dos colas de mensajes, sin memoria compartida ni semáforos.

2. Compilación y ejecución

Ambos programas se compilan con la librería de extensiones de tiempo real (-lrt), necesaria para las funciones mq_open, mq_send, mq_receive, mq_close y mq_unlink:

```
gcc -o prod prod.c -lrt
```

```
gcc -o cons cons.c -lrt
```

La ejecución requiere dos terminales simultáneas. El productor debe arrancar primero, ya que es el encargado de crear las colas (O_CREAT). El consumidor las abre sin crearlas:

```
Terminal 1: ./prod archivoProd.txt 5000
```

```
Terminal 2: ./cons archivoCons.txt 5000
```

El primer argumento es el fichero (de entrada para el productor, de salida para el consumidor) y el segundo es el valor máximo T en microsegundos para la espera aleatoria con usleep(). El fichero de salida del consumidor se crea vacío automáticamente al abrirse con fopen en modo escritura.

3. Diseño de la solución

3.1. Esquema de dos colas

Se utilizan dos colas de tamaño MAX_BUFFER=5: **/ALMACEN1** transporta items del productor al consumidor, y **/ALMACEN2** transporta mensajes vacíos (créditos) del consumidor al productor. Ambos procesos abren las dos colas: el productor escribe en /ALMACEN1 (O_WRONLY) y lee de /ALMACEN2 (O_RDONLY); el consumidor hace lo inverso.

3.2. Control de flujo

Al arrancar, el consumidor envía 5 mensajes vacíos por /ALMACEN2. Cada mensaje vacío es como una especie de crédito que autoriza al productor a enviar un item. Antes de cada envío, el productor ejecuta mq_receive(almacen2, ...) para consumir un crédito; si no hay créditos disponibles (buffer lleno), se bloquea automáticamente. Análogamente, el consumidor se bloquea en mq_receive(almacen1, ...) cuando no hay items (buffer vacío). Tras consumir un item, el consumidor devuelve un crédito con mq_send(almacen2, ...).

Este mecanismo reemplaza los tres semáforos de prácticas anteriores: mq_receive(almacen2) equivale a sem_wait(vacias), mq_receive(almacen1) equivale a sem_wait(llenas), y el semáforo mutex desaparece.

3.3. Detección de fin de fichero

La función produce_item() recoge el retorno de fgetc() en una variable int para comparar correctamente con EOF. Cuando se alcanza el final del fichero, devuelve el carácter '\0' como marca de fin. El productor detecta este valor, lo envía por /ALMACEN1 y sale del bucle. El consumidor, al

recibir '\0', interpreta la señal de terminación y finaliza también. La limpieza de las colas (mq_unlink) la realiza el consumidor al salir.

4. Resultados

Tras la ejecución, ambos procesos imprimen mensajes de seguimiento para posibles bugs y al llegar al final del archivo se imprime el conteo de vocales (A,B,C,D,E). Si el paso de vocales fue correcta, los conteos del productor y del consumidor coinciden exactamente, y el fichero de salida del consumidor es una copia idéntica del fichero de entrada del productor.

```
[Productor] Fin de fichero, señal enviada.  
[-]Productor: Vocales contadas: A:6 E:6 I:6 O:6 U:6 [Consumidor] Escritura finalizada. Vocales: A:6 E:6 I:6 O:6 U:6
```

[Productor] Iniciado. Leyendo de 'archivoProd.txt' con T=5000 us	[Consumidor] Iniciado. Escribiendo en 'archivoCons.txt' con t=5000 us
[Productor] Enviado: 'a'	[Consumidor] Recibido: 'a'
[Productor] Enviado: 'e'	[Consumidor] Recibido: 'e'
[Productor] Enviado: 'i'	[Consumidor] Recibido: 'i'
[Productor] Enviado: 'o'	[Consumidor] Recibido: 'o'
[Productor] Enviado: 'u'	[Consumidor] Recibido: 'u'
[Productor] Enviado: ' '	[Consumidor] Recibido: ' '

5. Conclusiones

Se verificó el comportamiento con distintos valores de T. Con T bajos ambos procesos terminan casi instantáneamente; con T altos se observa claramente el bloqueo del productor cuando agota los créditos y el bloqueo del consumidor cuando no hay items, confirmando que el sistema se autorregula correctamente sin riesgo de desbordamiento ni inanición.

La solución con paso de mensajes resulta más compacta y menos propensa a errores que la basada en semáforos y memoria compartida: se eliminan la gestión explícita del mutex, las llamadas a shm_open/mmap, y las condiciones de carrera derivadas del acceso directo a memoria.